

第 2 週：架構、資料型別與運算子

本週一句話：Verilog 的「數字」不只有值，還有寬度；運算子有五類，長得像但意義完全不同。

本週指令

雙擊 `env.bat` 開好環境（提示字元要有 `[OSS CAD Suite]`），然後：

```
cd week02_types_ops
```

每個範例都是這三條（外加一條檢查），把 `01_vectors` 換成你要跑的名字：

```
iverilog -o sim.out 01_vectors.v 01_vectors_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_vectors.gtkw
```

`echo %ERRORLEVEL%` 印 0 才是編譯成功——失敗時 iverilog 常常一個字都不印。

本週範例名稱：

- `01_vectors`
- `02_operators`
- `03_concat`
- `04_mux`
- `05_xz`
- `06_operators_all` ← 運算子全集（查表用）

下面每個範例的段落，都直接附了它自己的四行指令。

學完你要會什麼

- [] 會寫 `8'b1010_0101`、`8'hA5`、`8'd165` 三種等價寫法
 - [] 會做位元切片 `a[7:4]` 和串接 `{a, b}`
 - [] 分得出 `&` (位元) 和 `&&` (邏輯) 的差別
 - [] 知道縮減運算子 `^a` 在算什麼
 - [] ★ 看到波形上的 `x` 和 `z` 知道是什麼意思 (除錯關鍵)
-

一、數字怎麼寫

```
8'b1010_0101    // 二進位    (底線只是給人看，電腦忽略)
8'hA5            // 十六進位
8'd165           // 十進位
8'o245           // 八進位
```

四個都是同一個值 165。

格式固定是：

```
<幾位元> ' <進位> <值>
  8      '   h    A5
```

[!WARNING] 位元寬度非常重要。 `4'd15` 和 `8'd15` 值一樣，但一個是 4 條線、一個是 8 條線。寬度不夠會被無聲截斷——這是初學最常見的 bug。

```
wire [3:0] x = 8'd200;    // 200 裝不進 4 位元 → 只留低 4 位 = 8
```

二、切片與串接

寫法	意思
<code>a[7:4]</code>	取第 7 到第 4 位 (4 條線)
<code>a[0]</code>	取第 0 位 (1 條線 , 不用冒號)
<code>{a, b}</code>	把 a、b 兩束線接成一束 (a 在高位)
<code>{2{a}}</code>	把 a 複製兩次
<code>{{4{a[3]}}, a}</code>	符號延伸 (把最高位複製 4 次接在前面)

[!NOTE] 大括號 `{}` 在 Verilog 裡是「把線黏在一起」, 不是其他語言的程式區塊。程式區塊用 `begin / end`。

三、五類運算子 (最容易搞混的地方)

類別	符號	輸出寬度	說明
算術	<code>+ - * / %</code>	跟輸入一樣或更寬	會被綜合成加法器/乘法器
位元	<code>& ^ ~</code>	跟輸入一樣寬	一位一位分別做
邏輯	<code>&& !</code>	永遠 1 位元	把整條線當成一個真假值
關係	<code>== != < > <= >=</code>	永遠 1 位元	比較
縮減	<code>&a a ^a ~&a ~ a ~^a</code>	永遠 1 位元	把一整條線縮成一個位元
位移	<code><< >> <<< >>></code>	跟輸入一樣寬	左移 = 乘 2 , 右移 = 除 2 (有號數要用 <code>>>></code>)

★ 最重要的一組對照

```
a = 4'b1100;
b = 4'b1010;

a & b    = 4'b1000    // 位元 AND：4 個 AND 閘，輸出 4 位元
a && b   = 1'b1        // 邏輯 AND：a 非零 且 b 非零 → 1 位元
&a       = 1'b0        // 縮減 AND：a[3]&a[2]&a[1]&a[0]
```

三個都用到 `&`，意義完全不同。這是本週最需要記熟的。

★★ 縮減運算子的規則 (Python 沒有這種東西，要重新學)

縮減運算子只有一個運算元，寫在變數前面：`&a`、`|a`、`^a`。

它做的事是：把符號插到「同一條線的每個位元中間」，全部算成一個位元。

```
a = 4'b1100;    // a[3]=1 a[2]=1 a[1]=0 a[0]=0

&a  → 1 & 1 & 0 & 0 = 0    // 把 & 插到每個位元中間
|a  → 1 | 1 | 0 | 0 = 1
^a  → 1 ^ 1 ^ 0 ^ 0 = 0
```

[!IMPORTANT] 這跟位元運算子長得一樣但位置不同：`a & b` 是兩個運算元 (中間)，`&a` 是一個運算元 (前面)。看到符號前面沒東西，就是縮減。

三個基本的各自在問什麼

寫法	展開	白話：它在問什麼	什麼時候用
<code>&a</code>	<code>a[3]&a[2]&a[1]&a[0]</code>	「是不是全部都是 1？」	計數器數到滿、位址全中
<code> a</code>	<code>a[3] a[2] a[1] a[0]</code>	「有沒有任何一個 1？」	★ 判斷 a 是不是 0 (<code> a==0</code> 就是 a 全零)
<code>^a</code>	<code>a[3]^a[2]^a[1]^a[0]</code>	「1 的個數是不是奇數？」	★ 同位元檢查 parity

帶 `~` 的三個 = 把結果反相

先縮減，再整個反相，不是先反相再縮減：

```
~&a = ~(&a) = ~0 = 1    // 縮減 NAND
~|a = ~(|a) = ~1 = 0    // 縮減 NOR
~^a = ~(^a) = ~0 = 1    // 縮減 XNOR (1 的個數是偶數 → 1)
```

`a = 1100` 的完整六個答案 (`06_operators_all` 跑出來就是這樣)：

	<code>&a</code>	<code> a</code>	<code>^a</code>	<code>~&a</code>	<code>~ a</code>	<code>~^a</code>
<code>a = 1100</code>	0	1	0	1	0	1

為什麼 `^a` 就是 parity

XOR 有個特性：接上一個 1 就翻面，接上 0 沒事。所以一路 XOR 下來，等於在數「翻了幾次面」——翻奇數次是 1，偶數次是 0。

```
wire parity = ^data;    // 8 位元的同位元，一行就好
```

這就是資料傳輸偵錯最基本的手段：送 9 位元 (8 資料 + 1 parity)，收到後再算一次 `^` 比對，對不上就是中途有位元被弄反了。

[!TIP] `|a` 拿來判斷「a 是不是 0」比 `a == 0` 更省。`a == 0` 要一個比較器，`|a` 只要一棵 OR 樹，綜合出來面積更小。這是實務上很常見的寫法。

► 進階：`~^a` 為什麼不是「一路 XNOR 接下去」(點開看，初學可跳過)

★★ 位移運算子的規則 —— `>>` 和 `>>>` 差在哪

數學上「二進位向右移 1 格」=「除以 2」。但這件事只有無號數成立。

問題出在哪

```
a = 4'b1100;
```

這串 `1100` 到底是多少？看你把它當成什麼：

- 當成無號數 → 就是 12

- 當成**有號數**（二補數）→ 最高位是 1 代表負數 → 是 -4

同一串位元，兩種讀法。而右移要補什麼，就取決於這個。

兩種右移

	左邊補什麼	1100 右移 1 格	讀成有號數	對不對
>> 邏輯右移	永遠補 0	0110	+6	X -4 除以 2 怎麼會變 +6
>>> 算術右移	補符號位（這裡是 1）	1110	-2	✓ $-4 \div 2 = -2$

>>> 存在的唯一理由，就是讓「負數除以 2」在硬體上還是對的。補符號位這個動作，等於在說「它本來是負的，移完還是負的」。

[!WARNING] >>> 要 **signed** 才有效。這是最常踩的坑——光寫 >>> 沒有用，運算元本身必須是有號的：

```
wire      [3:0] u = 4'b1100;    // 沒宣告 signed
wire signed [3:0] s = 4'b1100;    // 有宣告

u >>> 1    = 0110    ← 跟 >> 一模一樣，補 0，白寫了
s >>> 1    = 1110    ← 這才是算術右移
```

編譯器不會警告你，波形也不會變紅——它就只是安靜地算錯。

左移只有一種

<< 和 <<< 結果**永遠一樣**。左移是往低位補 0，不管有號無號都一樣，所以 <<< 只是為了對稱才存在的。

```
1100 << 1 = 1000    // 有號： $-4 \times 2 = -8$  ✓
0110 << 1 = 1100    // 有號： $+6 \times 2 = -4$  X 溢位了！
```

左移會**溢位**：4 位元只裝得下 -8~+7，超過就繞回去。移出去的那個位元是直接丟掉的，不會有進位。

► 進階：`>>> 1` 其實不完全等於 `/2` (點開看)

四、★ 四值邏輯 0 / 1 / x / z

Verilog 的線不是只有 0 和 1，還有兩個狀態：

值	意思	什麼時候出現	波形上長怎樣
0	低電位	正常	低的線
1	高電位	正常	高的線
x	不知道	reg 沒給初值、reset 沒接、兩個東西搶同一條線	紅色
z	高阻抗 (浮空)	線沒接、三態緩衝器關掉	中間高度的線

[!!IMPORTANT] 這是你以後最重要的除錯線索。

- 波形一片紅色 x → 通常是忘了給初值，或 reset 沒接好
- 波形出現中線 z → 通常是忘了接線，或三態沒開 enable

而且 x 和 z 會傳染——一個 x 進到運算裡，整串結果都變 x。所以要往最左邊、最上游找源頭，不是看最後那條。

五、本週六個範例

```
cd week02_types_ops
```

01_vectors — 向量與切片

```
iverilog -o sim.out 01_vectors.v 01_vectors_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_vectors.gtkw
```

看什麼：波形上 `data` 是一整條 8 位元匯流排，而 `high`、`low`、`msb` 是從它切出來的。`data` 一變，切片同時跟著變。終端機會印四種數字寫法的對照表。

📁 02_operators — 運算子大集合

```
iverilog -o sim.out 02_operators.v 02_operators_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_operators.gtkw
```

看什麼：18 條輸出線，同一組 `a`、`b` 餵進去，各種運算子的結果一次全看到。重點盯 `o_and`（4 位元）和 `o_land`（1 位元）的寬度差別。

📁 03_concat — 串接與複製

```
iverilog -o sim.out 03_concat.v 03_concat_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_concat.gtkw
```

看什麼：`{a,b}` 和 `{b,a}` 在波形上剛好高低對調。最後有符號延伸的說明——為什麼 `4'b1000`（-8）延伸成 8 位元要補 1 不能補 0。

📁 04_mux — 多工器

```
iverilog -o sim.out 04_mux.v 04_mux_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_mux.gtkw
```

看什麼：`sel` 一變，`y4` 立刻跳到對應那一路的值（A/B/C/D 很好認）。多工器是數位電路最常用的零件之一，第 3 週的 `case` 就是它的另一種寫法。

📁 05_xz — x 和 z ★ 本週重點

```
iverilog -o sim.out 05_xz.v 05_xz_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 05_xz.gtkw
```


看什麼：★ 波形上親眼看到 **z**（三態關掉時）和 **x**（reg 沒給初值時）。

實測輸出：

```
enable=1 data=1 → tri_out = 1    (正常輸出)
enable=0         → tri_out = z    ★ z! 線被放掉了
never_set        = xxxx          ★ x! reg 宣告了卻沒給值
```

這一個範例的除錯價值，比前面四個加起來還高。

📁 06_operators_all — 運算子全集 📖 查表用

```
iverilog -o sim.out 06_operators_all.v 06_operators_all_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 06_operators_all.gtkw
```

02_operators 每一類只挑了代表。這一份把上面表格裡的每一個運算子都跑一次，共 31 個，分成六組印出來，忘記哪個是什麼就跑一次查。

多出來的是： `/` `%` `**` 單元 `-` `·` `~^` `·` `||` `·` `!=` `>` `<=` `>=` `===` `!=` `·`
`~&a` `~|a` `~^a` `·` `<<<` `>>>`

六組測試向量，每組把 31 個運算子全部印一遍：

#	a	b	為什麼挑這組
①	1100 (12)	1010 (10)	上面表格用的那組
②	1111 (15)	0011 (3)	除得盡：15/3 = 5 餘 0
③	0000 (0)	0101 (5)	a=0，邏輯與縮減的差別最明顯
④	0110 (6)	0000 (0)	★ b=0 → <code>/</code> 和 <code>%</code> 整條變 x
⑤	0111 (7)	0111 (7)	兩邊相等
⑥	10x1	10x1	★ 含 x → <code>==</code> 給 x， <code>===</code> 敢說 1

三個一定要看的地方（跑完最後會單獨再印一次）：

```
a=1100 b=1010
a & b   = 1000    位元 AND：4 個 AND 閘，輸出 4 位元
a && b  = 1        邏輯 AND：兩邊都非零，輸出 1 位元
&a      = 0        縮減 AND：a[3]&a[2]&a[1]&a[0]
```

```
a >> 1 = 0110    無號 12 → 6，左邊補 0
a >>> 1 = 1110    有號 -4 → -2，左邊補符號位 ★ 只有 signed 才看得出差別
```

b=0000 時 a/b = xxxx a%b = xxxx ← 除以 0 沒有定義，不是 0

[!WARNING] === 和 !== 只能模擬，不能合成——真實的線上沒有 x 這種電壓。它們只在 testbench 裡用來檢查「有沒有訊號忘了接」。

六、練習題

練習 1

寫一個 `parity` 模組：輸入 8 位元，輸出 1 位元的偶同位。提示：一行就好，用縮減運算子。

練習 2

寫一個 `rotate_left`：8 位元輸入，循環左移 1 位（最高位繞回最低位，不是補 0）。

提示：`{a[6:0], a[7]}` —— 想清楚為什麼這樣寫就對了。

練習 3 (思考)

下面這行有什麼問題？

```
wire [3:0] sum;
assign sum = 4'd9 + 4'd8;
```

► 看答案

七、本週檢核

- [] 五個範例都跑過，波形都看過
 - [] 說得出 `a&b`、`a&&b`、`&a` 三者的差別
 - [] 在 `05_xz` 的波形上找得到 `z` 那一段
 - [] 練習 1、2 寫出來
 - [] 練習 3 答對 (沒答對就再讀一次「位元寬度」那段)
-

附註：關於 `sim` 這個指令

你可能在別的地方看過我寫的 `sim` 範例名稱。那是我自己做的批次檔 (`sim.bat`)，不是 Verilog 或 iverilog 的標準指令，教科書上查不到。

它做的事就是把上面那四行包起來、順便幫你檢查結束碼。用不用都可以，也可以刪掉。這份 README 裡給的全部都是原始指令。